RELEASE NOTES

EE362 Project 1 · 변경 이력

버전 / 날짜	변경 사항
R1.3 **2026.10.23**	입력·출력 delay의 -min/-max 모두 1.00 ns 동기 active-high RESET 사용 의무화 합성 후 정상 모드 STA에 set_case_analysis 0 적용 PPTX·PDF·SDC를 같은 배포판으로 갱신
R1.2 / R1.1 **2026.10.20**	R1.2 PPA 25점·발표 10점 / R1.1 SRAM 추정치·포트 계약

다음 개정 시: 버전·날짜·변경 사항·영향 파일·검증 결과를 이 표에 추가

주의: 정적전력 계수는 EDK 실측 누설전력이 아니라, SAED32 SRAM 면적에 비례시킨 고정 채점 정책임.

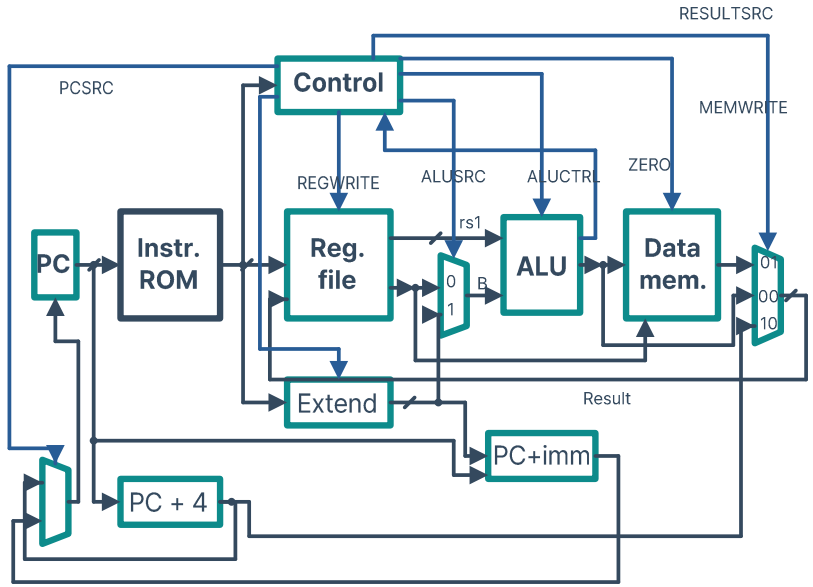
Project 1
INT8 선형계층

디지털집적회로모델링실험 EE362

프로젝트 명세 및 실습 안내

학생 배포본

명세, PPA 상대평가, 어셈블리 사용법
파일별 역할과 제출 방법



기준 프로그램 실행부터 최적화 결과 비교까지

학생 배포본 · 2026.10.23

계산 명세

입력 8개, 출력 4개

가중치 32개와 bias 4개를 이용한 정수 내적

고정 계산 범위

signed INT8 곱을 INT32로 누산. N=8, M=4 고정

과제의 범위

activation·재양자화·학습 제외

선형계층의 누산

$$Y[j] = BIAS[j] + \Sigma\, X[i] \times W[j][i]$$

$$i = 0 \ldots 7,\quad j = 0 \ldots 3$$

자료형과 입력 범위

항목	개수	범위
X	8	−128 … 127
W	4 × 8	−127 … 127
BIAS	4	-2^{30} … 2^{30}
Y	4	signed INT32

각 출력은 가중치 한 행과 입력 벡터의 내적

명세 확인 · README_KO.md

구현 범위와 제공 범위

학생 구현 범위

CONTROLLER·TOP 연결. 선택 RV32I는 CPU 내부 변경 허용

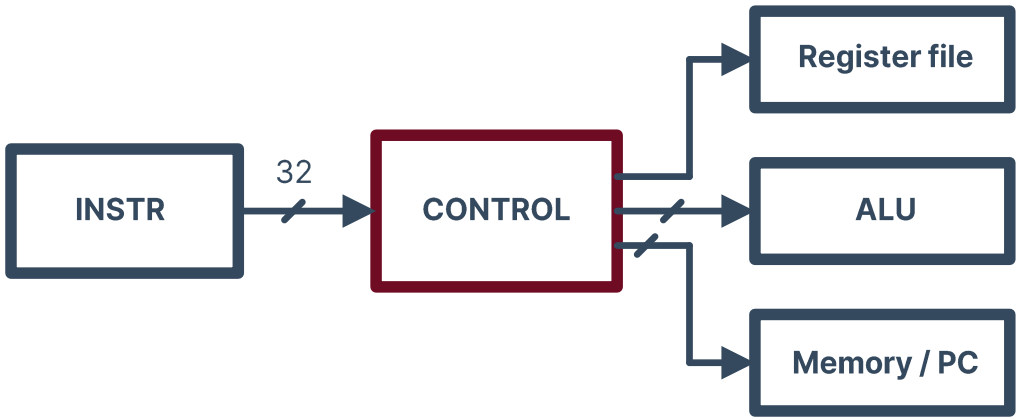
제공 예정 데이터패스

PC·레지스터 파일·ALU는 합성 RTL 제공. IMEM/DMEM은 기능 모델이며 SRAM 비용은 PPA에 별도 가산

현재 ZIP의 범위

기본 8명령 도구만 포함. 합성 CPU·확장 검증기는 별도 배포

제어 신호의 연결 대상



명령어 비트로 데이터패스 동작 선택

현재 tb_behavior_cpu.sv는 기능 실행 모델
IMEM/DMEM은 CPU 합성에서 제외, SRAM 상당 비용은 가산

제어 신호와 데이터패스 연결의 이해가 기본 과제

구현 범위안 · CPU starter 배포 후 적용

기본 ISA와 선택 확장

필수 기본 ISA

ADD, SUB, AND, ADDI, LW, SW, BEQ, JAL

선택 RV32I

Harris 표 B.1의 32비트 정수 명령. M 확장 제외

주소와 x0

PC는 바이트 주소, 일반 진행 +4. x0 읽기는 항상 0

필수 8개 명령의 역할

명령	역할
ADD / SUB	덧셈 / 뺄셈
AND	비트 단위 AND
ADDI	부호 확장 immediate 덧셈
LW / SW	32비트 word 읽기 / 쓰기
BEQ	두 레지스터가 같으면 분기
JAL	점프 및 PC+4 link 기록

선택 명령은 표준 동작을 검증하고 목록을 제출

현재 ZIP은 기본 8개만 지원 · M/RV64/F/D/C/CSR 제외

INT8 값의 저장 규칙

값의 범위와 저장 크기

INT8 값 하나를 32비트 word 하나에 부호 확장하여 저장

음수 표현

최상위 부호 비트의 반복. −3의 저장값은 fffffffd

가중치 배열

W[j][i]를 행 우선 순서로 저장. 다음 원소 주소는 +4

값과 메모리 표현

값	32비트 word	해석
127	0000007f	+127
−3	fffffffd	−3
−128	ffffff80	−128
잘못된 −3	000000fd	+253

메모리 용량

DMEM: 512 word = 2,048 byte
현재 계약은 INT8 packing 미사용

작은 숫자 범위와 실제 저장 크기의 구분

입력 확인 · cases/*_dmem.hex

Configuration과 메모리 계약

고정 시작 위치

0x300에서 설정 10개를 읽어 주소와 크기 확인

실행마다 달라지는 배치

X/W/BIAS/Y/SCRATCH의 시작 주소를 코드에 고정하지 않음

쓰기 허용 영역

Y, SCRATCH, DONE만 사용. 입력과 설정 영역 보존

0x300 configuration block

주소	필드	의미
0x300	X_BASE	입력 시작
0x304	W_BASE	가중치 시작
0x308	BIAS_BASE	bias 시작
0x30C	Y_BASE	출력 시작
0x310	SCRATCH_BASE	중간 결과 시작
0x314	N	8
0x318	M	4
0x31C	DONE_ADDR	제공 fixture: 0x3F0
0x320	DONE_TOKEN	실행별 0이 아닌 token
0x324	SIGN_MASK	0x80000000

DMEM 512×32 고정. 과거 GEMM 설정 파일 사용 금지

명세 확인 · 새로운 INT8 메모리 계약

느린 기준 프로그램의 구조

1단계: 곱 계산

MUL 대신 |W|회 덧셈. 중간 곱 32개를 SCRATCH에 저장

2단계: 누산

중간 곱을 다시 읽어 bias와 합산 후 Y 4개 기록

개선 가능 지점

반복 연산, 중간 저장, 반복 읽기에 따른 실행량 증가

실제 소스의 반복 덧셈 부분

```
multiply_setup:
    addi x8, x0, 0

multiply_loop:
    beq  x7, x0, multiply_done
    add  x8, x8, x9
    addi x7, x7, -1
    jal  x0, multiply_loop

multiply_done:
    sw   x8, 0(x3)
    addi x3, x3, 4
```

x7: 반복 횟수, x9: 부호를 조정한 X

실행량을 줄일 여지가 큰 기준 프로그램

코드 탐색 · baseline_int8_fc_slow.s

기능 검사와 완료 조건

결과 확인

모든 Y 4개가 정답과 동일. 음수와 0 가중치 포함

메모리 보존

허용 영역 외 쓰기 금지. 주소 배치가 바뀌어도 동일 동작

완료 시점

Y 기록 완료 후 DONE_ADDR에 해당 실행의 token 기록

실제 소스의 종료 부분

```
accumulate_row_done:
    sw   x10, 0(x24)
    # 다음 출력으로 이동하는 코드 생략

done:
    sw   x29, 0(x28)

park:
    jal  x0, park
```

완료 검사

DONE store 이벤트까지 측정
이후 park 반복은 측정에서 제외

정답 출력과 정상 완료 순서를 함께 확인

기능 기준 · cases/*.json과 *_expected.hex

PPA 상대평가: 25점

평가 관문: PASS / FAIL

기능·제어·연결·재현·STA 확인

상대평가: PPA 25점

성능:면적:전력 = 2:1:1

관문 미통과: 0점

PASS 팀 뒤 공동 최하위

PPA 관문과 별도 발표

평가 항목	판정	적용
기능 정확성	PASS	공개·숨김 24개 전부 통과
제어유닛·연결	PASS	ISA·고정 포트·안전 decode
재현·개선·STA	PASS	재실행·비교 근거·timing
발표 피어리뷰	별도	프로젝트 1 발표 10점
PPA 상대평가	25	성능 12.5 / 면적 6.25 / 전력 6.25
합계	25	관문 통과팀끼리 PPA 비교

PPA 관문 중 하나라도 FAIL이면 비교 제외

1주차 배점 · P1 PPA 25점, 발표 피어리뷰 10점

PPA 참가를 위한 PASS / FAIL

기능: PASS / FAIL

공개·숨김 24개 모두 정답·쓰기·DONE 충족

제어유닛·연결: PASS / FAIL

고정 포트·기본 ISA·safe decode 충족

재현·개선 근거: PASS / FAIL

깨끗한 재실행과 변경 전후 T/A/P 확인

필수 통과 항목

평가	세부 확인
기능 테스트	24개 case의 정답·쓰기 범위·DONE 순서 전부 PASS
제어유닛	명령별 제어 신호와 기본값 불법 명령의 안전한 처리
TOP 연결	포트 이름의 명시적 연결 한 줄당 한 핀·포트·넷
재실행	깨끗한 작업 폴더에서 동일 결과 재생성
개선 근거	변경 전후 T/A/P 비교와 소스·로그 일치

기능 어느 관문이든 FAIL이면 PPA 0점·공동 최하위

조교 확인 · 공식 채점 덱은 별도 제작 대상

PPA 측정 조건과 SDC

평가 참가 요건

기능·제어·연결·재현·근거·합성·timing 전부 PASS

공통 측정 환경

SAED32 SS 0.95 V, 125 °C. 팀별 f만 변경

비교 항목

T·A·P 측정. HEX word 수로 IMEM SRAM 용량을 결정

고정 SDC 조건

항목	고정값
CLK	T = 1000 / f_MHz ns setup/hold uncertainty: 0.10 / 0.05 ns clock transition: 0.08 ns
입력	-min = -max = 1.00 ns input transition: 0.10 ns
출력	-min = -max = 1.00 ns load: 20 fF · max transition: 0.25 ns
RESET	동기 active-high (1 = assert) 정상 모드 STA에서는 0으로 고정
적용 시점	set_case_analysis는 합성 후 STA 전용 RESET 기능은 시뮬레이션으로 별도 검사

set_case_analysis 0 [get_ports RESET]

합성 중 case analysis 적용 금지 · PPA 평가는 동일 SDC로 비교

PPA 상대평가 25점 산식

통과팀만 상대평가

T/A/P별 최솟값, 배점 2:1:1

공통 측정 조건

같은 corner·case. FAIL이면 0점·공동 최하위

발표는 피어리뷰

프로젝트 1 발표 10점, PPA와 별도

25점 상대 점수

$$S = 12.5 \times \min(1, T_{best} / T_i) + 6.25 \times \min(1, A_{best} / A_i) + 6.25 \times \min(1, P_{best} / P_i)$$

작은 값일수록 높은 점수
각 best는 평가 참가 요건 통과팀 기준

배점 비율과 통과 판정

성능 T	면적 A	전력 P	총점	FAIL
12.5	6.25	6.25	25	0

e-Campus 100점 환산 = S × 100 / 25

설계 개선의 근거와 결과를 함께 평가

PPA 25점 · 발표 피어리뷰 10점은 별도

발표 피어리뷰 평가 요소

프로젝트별 성적 구성

PPA 25점과 발표 피어리뷰 10점

동료 평가의 대상

설명·검증 근거·질의응답

점수의 경계

PPA 수치 자체는 재채점하지 않음

동료 평가자가 확인할 근거

평가 요소	자료	동료가 확인할 내용
설계 이해	구조도	고정 인터페이스와 제어 흐름 설명
기능 증거	로그	Y·DONE 결과와 검사 범위 설명
PPA 비교	표	같은 조건의 기준본·개선본 T/A/P
개선 근거	코드	변경 이유와 성능·면적·전력 영향
재현 방법	절차	파일·명령·조건을 따라 실행 가능
질의응답	답변	구현의 한계와 trade-off 설명

자료·시연·답변을 근거로 평가

1주차 배점 · 프로젝트 명세 50점, 피어리뷰 20점

어셈블리와 HEX, LST의 관계

어셈블리 원본 .s

레지스터 연산, 분기 label, 한글 주석을 편집하는 파일

기계어 .hex

IMEM에 입력할 32비트 명령. 한 줄이 한 word

대조표 .lst

바이트 PC와 기계어를 원본 명령과 대조하는 파일

원본 코드 읽기

```
start:
    addi x20, x0, 768
    lw   x21, 0(x20)
    lw   x22, 4(x20)

# x3가 가리키는 word에 기록
    sw   x8, 0(x3)
    addi x3, x3, 4
```

확인 포인트

label은 주소 계산에 사용
주석과 label은 별도 명령 word 미생성
원본 수정 후 HEX/LST 재생성 필요

학생은 .s를 수정하고 도구로 HEX/LST 생성

실습 1 · baseline_int8_fc_slow.s 열기

실행 1: 압축 해제와 인코딩

ZIP을 올려둔 폴더

같은 이름의 폴더가 없는 새 작업 위치에서 압축 해제

작업 위치 확인

폴더 이동 후 pwd로 패키지 루트 확인

원본 인코딩

.s에서 .hex와 .lst 생성. 아래 명령을 위에서부터 실행

Linux 서버에서 순서대로 실행

```
# 배포 ZIP 압축 해제
unzip Project1_INT8_Baseline_20261007.zip

# 기준본 폴더로 이동
cd Project1_INT8_Baseline_20261007

# 현재 작업 위치 확인
pwd

# 원본을 기계어와 대조표로 변환
python3 tools/rv32i_assembler.py \
  baseline_int8_fc_slow.s \
  -o baseline_int8_fc_slow.hex \
  --listing baseline_int8_fc_slow.lst
```

줄 끝의 역슬래시는 다음 줄로 이어지는 한 명령

실습 2 · 새 폴더에서 압축 해제 후 실행

실행 2: Python 기능 확인

기본 예제 실행

기계어를 기능 모델에서 실행하여 Y와 완료 확인

전체 기준본 회귀

제공된 공개 입력 16개와 주소 배치를 모두 검사

기대 결과

demo의 Y는 [6, 4, 10, 90]. 전체 결과는 16/16 PASS

앞 페이지에 이어서 실행

```
# 기본 demo 기능 모델 실행
python3 tools/reference_run.py

# 제공 기준본 16개 입력 회귀
python3 verify_all.py

# 기대 결과의 핵심 값
#  demo:  Y=[6, 4, 10, 90]
# demo: 명령 900, LD 110, ST 37
# 마지막: TOTAL PASS 16/16
```

사용 범위

verify_all.py는 고정 기준본 검사
최적화 프로그램의 공식 채점기로 사용 금지

먼저 제공 기준본의 PASS를 재현한 뒤 코드 수정

실습 3 · 출력과 기능 로그 확인

실행 3: VCS 컴파일과 실행

폴더 생성

실행물은 build, 실행 기록은 logs에 보관

VCS 컴파일

TB_BEHAVIOR_CPU를 top으로 지정하여 기능 모델 컴파일

demo 실행

기본 HEX와 입력으로 실행 후 로그의 PASS 확인

패키지 루트에서 순서대로 실행

```
# 실행물과 로그 폴더 생성
mkdir -p build
mkdir -p logs

# behavioral 실행 모델 컴파일
vcs -full64 -sverilog \
  -top TB_BEHAVIOR_CPU \
  sim/tb_behavior_cpu.sv \
  -o build/simv -l logs/compile.log

# 기본 demo 실행
./build/simv -l logs/demo.log

# 실행 결과 확인
cat logs/demo.log
```

VCS 실행 모델은 기능 확인용이며 합성용 CPU와 구분

실습 4 · VCS wrapper는 일반 옵션 전달 기준

파일 설명: 프로그램과 입력

수정할 원본

기준 .s를 복사해 개선 프로그램 작성

입력과 정답 분리

DMEM 이미지로 실행. expected 파일은 검사 모델만 참조

입력 배치 유지

case별 설정 주소와 token 사용. 정답 파일 수정 금지

패키지 루트와 cases/

파일	역할
baseline_int8_fc_slow.s	한글 주석을 포함한 원본
baseline_int8_fc_slow.hex	32비트 IMEM 명령어
baseline_int8_fc_slow.lst	PC·기계어·원본 대조표
cases/*.json	X/W/bias·주소 배치·정답
cases/*_dmem.hex	512-word 초기 DMEM
cases/*_expected.hex	Y 4개 정답 이미지

프로그램은 입력과 설정만 읽고 직접 Y를 계산

파일 탐색 · cases/demo.json 먼저 확인

파일 설명: 실행 도구와 검증

도구의 역할

인코딩, Python 기능 실행, 독립 HDL 실행을 각각 수행

기준본 검사 범위

verify_all과 verify_rtl은 고정 실행량까지 검사

최적화 검사 분리

정답 검사와 실행량·PPA 측정을 분리한 공식 덱 필요

tools/, sim/와 회귀 도구

파일	역할
tools/rv32i_assembler.py	8명령 인코딩
tools/reference_run.py	HEX 기능 실행 모델
tools/make_cases.py	고정 seed의 공개 입력 생성
sim/tb_behavior_cpu.sv	simulation-only 실행 모델
verify_all.py	Python 기준본 전체 회귀
verify_rtl.py	VCS / Icarus 기준본 회귀

고정 실행량 검사 파일은 최적화 채점기로 사용 금지

파일 탐색 · verify_all.py와 verify_rtl.py

파일 설명: 문서와 실행 기록

사용 설명 문서

README와 PPA 주의사항을 먼저 읽고 범위 확인

검증 기록

JSON은 제공 기준본의 로컬 실행 기록. PPA 보고서와 구분

파일 무결성

배포 파일 목록·해시로 기준본의 변경 여부 확인

문서와 JSON 기록

파일	내용
README_KO.md	계산·메모리 계약과 실행 순서
PPA_비교_주의사항_KO.md	환경·메모리 비용·측정 조건
FUNCTIONAL_VALIDATION.json	Python 공개 16개 실행 기록
HDL_VALIDATION.json	Icarus 기능 모델 실행 기록
MANIFEST.json	배포 파일 목록과 해시
build/와 logs/	실행 시 생성하는 개인 작업물

PASS 기록만으로 실제 합성 CPU의 동작을 보장하지 않음

파일 탐색 · 제공 기록과 본인 실행 기록 구분

32 nm SRAM 메모리 비용

면적 가산

IMEM/DMEM 매크로 면적을 CPU cell area에 더함

동적·정적전력 가산

동적: f·메모리 접근 / 정적: 매크로 면적

추정 범위

정적전력은 수업용 추정치, 실측값 아님

SAED32 SRAM1RW 면적과 접근 에너지

메모리	용량	면적 [μm²]	E [pJ]
IMEM	64×32	8,713.50	2.23
IMEM	256×32	27,352.30	7.00
IMEM	512×32	50,667.68	12.97
DMEM	512×32	50,667.68	12.97

메모리 전체전력 [mW]

$$f_MHz/1000 \times [E_IMEM + 12.97 \times accesses/cycles] + 0.020 \times A_mem/1000$$

IMEM은 HEX 길이로 선택. DMEM은 512×32 고정

면적은 SAED32 값 · 정적 계수는 채점 정책

개선 실험과 허용 변경 범위

첫 개선 후보

곱 계산 직후 누산하여 SCRATCH 저장·재읽기 감소

반복 제어 개선

루프 전개 등으로 분기 감소. IMEM HEX word 수 증가도 별도 확인

선택 RV32I 확장

Harris 표 B.1만 허용. M 확장 전체 제외

개선과 확인할 비용

개선	확인 사항
곱·누산 루프 결합	정답과 DONE 유지 SCRATCH 접근 감소량
루프 전개	명령 수와 IMEM HEX word 수 프로그램 크기·fetch 활동
선택 RV32I	표준 의미·단독 시험 사용 명령 목록 제출
공통 제한	정답 lookup·입력 특화 금지 Y와 완료 계약 유지

선택 확장은 구현·검증 후 사용. M 확장 사용은 실패

한 번에 한 변경을 적용하여 이득과 비용을 비교

설계 실험 · 기본 8개 필수, RV32I 선택 구현

제출물 구성안

소스와 생성물

RTL, 어셈블리, HEX/LST와 선택 ISA 시험을 함께 제출

실행 근거

기능·STA·CPU area/power와 f, IMEM word 수 제출

설계 설명

기준본 대비 변경 이유와 항목별 개선·손실 정리

팀별 제출 폴더 예시

```
TEAM_ID/
  rtl/
    controller.v
    top.v
  asm/
    optimized.s
    optimized.hex
    optimized.lst
  run/README_KO.md
  run/reproduce.sh
  results/functional.json
  results/cycles.csv
  results/area.rpt
  results/timing.rpt
  results/power.rpt
  report/project1.pptx
```

포트·선택 ISA·메모리 용량까지 재현 가능해야 함

제출안 · 최종 starter와 공식 실행 스크립트 기준

TOP 고정 포트 계약

모듈·포트 고정

TOP, CORE, CONTROLLER, IMEM, DMEM 이름·방향·폭 유지

예약 포트

TOP에 32비트 입력 20개와 출력 20개를 추가

채점 경계

포트 불일치·예약 입력 비영값 의존 시 실패

TOP 외부 인터페이스

구분	고정 이름·폭
입력	CLK, RESET: 각 1비트
예약 입력	RESERVED_IN0..19: 각 32비트
관찰 출력	PC, INSTR, ALURESULT, WRITEDATA: 각 32비트 MEMWRITE: 1비트
예약 출력	RESERVED_OUT0..19: 각 32비트, 0 고정
RESET·DONE	동기 active-high reset DONE_ADDR에 마지막 store

공식 TB: 예약 입력=0, 예약 출력=미연결 · 학생 수정 불가

인터페이스 계약 · 전체 선언은 별첨 명세 참조

부록 A: 내부 모듈 고정 포트

CONTROLLER

W07/W08 제어 신호의 이름·폭을 그대로 유지

CORE

CLK, RESET, INSTR, READDATA 입력; PC, MEMWRITE, ALURESULT, WRITEDATA 출력

IMEM/DMEM

W08의 ADDR·INSTR·WRITEDATA·READDATA 이름 유지

CONTROLLER 포트 계약

포트	방향·폭
INSTR	입력 32비트
ZERO	입력 1비트
REGWRITE / MEMWRITE / ALUSRC	출력 각 1비트
RESULTSRC / IMMSRC	출력 각 2비트
ALUCTRL	출력 3비트
PCSRC	출력 1비트
ILLEGAL	출력 1비트
추가 포트	허용하지 않음

명시적 한 줄 한 포트 연결. 이름·방향·폭 변경 금지

포트 계약 · TOP 예약 포트만 추가

부록 B: 기본 코드 기능 확인

기본 코드

기준본을 보존하고 optimized.s에서 변경

선택 확장

공식 도구 제공 후 RV32I 명령·단독 시험 제출

기능 검증

기본 8개는 아래 명령으로 demo 확인. 최종은 공식 덱 평가

기본 8명령용 현재 ZIP 실행 순서

```
# 개선 원본의 HEX와 LST 생성
python3 tools/rv32i_assembler.py \
  optimized.s -o optimized.hex \
  --listing optimized.lst

# 개선 HEX를 demo 입력에서 실행
python3 tools/reference_run.py \
  --program optimized.hex \
  --case cases/demo.json

# verify_all / verify_rtl은
# 기준본 전용 고정 실행량 검사
```

demo 통과만으로 전체 기능·PPA 통과 불가

최적화 프로그램에는 별도의 전체 기능 채점 덱 필요

현재 ZIP은 기본 8개만 조립·실행

부록 C: 오류 확인과 전체 회귀

명령어 해석 오류

선택 RV32I면 도구 지원 확인. 의사 명령은 제거

기준본 회귀 실패

소스 수정 후 인코딩 재실행. 실행량 고정 검사 여부 확인

VCS 실행 오류

패키지 루트, compile.log와 wrapper 옵션 전달 확인

기준본 전체 HDL 회귀

```
# 먼저 기준본 Python 회귀 확인
python3 verify_all.py

# 서버의 VCS wrapper로 전체 회귀
python3 verify_rtl.py --backend vcs

# VCS 대신 Icarus 사용 시 대체 명령
# python3 verify_rtl.py --backend iverilog
```

오류를 찾을 파일

인코딩 오류: .s와 .lst
기능 불일치: case JSON과 실행 로그
컴파일 오류: logs/compile.log

복사 명령의 작업 위치와 첫 오류부터 확인

기준본 회귀 · 서버 실행은 조교 사전 확인 필요

참고 자료와 적용 범위

수업 기준 파일

Project1_INT8_Baseline_20261007의 원본과 설명서

ISA 의미

Harris 부록 B 표 B.1의 RV32I만 선택. M 확장 제외

INT8 계산 배경

LiteRT 양자화 명세의 정수 내적 개념을 단순화

참고 자료

수업 배포 파일
README_KO.md
인터페이스·PPA 계약서 (2026-10-19)
baseline_int8_fc_slow.s
FUNCTIONAL_VALIDATION.json
HDL_VALIDATION.json

Harris 부록 B 표 B.1 / RISC-V RV32I 공식 명세
Google LiteRT INT8 양자화 명세

외부 자료의 상세 URL: 발표자 노트

CPU starter·공식 채점기와 일치 확인 후 학생 배포

학생 배포본 · 2026.10.23